What is a network?

clients, servers and the conversation between them

Watch the 8-min video

In one line: a network links devices so they can send messages; a client asks with a request, a server does the work and answers with a response, and every round trip costs time.

WORDS YOU'LL MEET TODAY

- **Network**: devices linked so they can send messages
- **Client**: the one who asks (your phone's app, a browser)
- **Request**: the question the client sends
- **Round trip**: a request going there + its response coming back
- **Timeout**: "I waited too long": the client gives up
- **Internet**: a network of networks
- **Server**: the one who answers: a computer that waits for requests
- **Response**: the answer that comes back
- **Millisecond (ms)**: a thousandth of a second
- **Retry**: send the same request again

1 The question: who talked to whom?

You open the Flipkart app, type *running shoes* and tap Search. Half a second later, a page full of shoes appears. But those shoes were never on your phone. Something far away sent them. This lesson follows that conversation, one step at a time.

A **network** is devices linked so they can send messages to each other. Your home Wi-Fi is a small one: it links your phone, laptop and TV. The **internet** is many networks joined together, a network of networks; a message to a computer in another city passes through several of them.

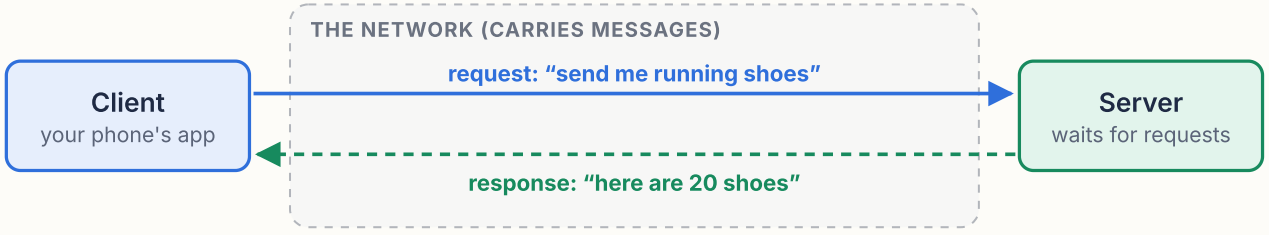
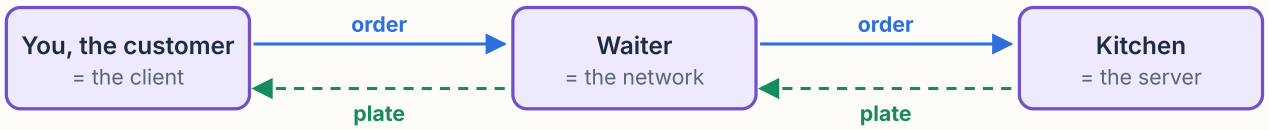


Figure 1. A network lets devices send messages. The client asks with a request; the server answers with a response.

2 An everyday picture first: a restaurant

You sit at a table, hungry. You don't walk into the kitchen; you call the waiter and order one masala dosa. The waiter carries the order to the kitchen, the kitchen cooks, and the waiter carries the plate back. You asked, and you got an answer.

- **You always speak first.** The kitchen never sends food nobody ordered.
- **One kitchen cooks for many tables** at the same time.



Order = request · plate of food = response

Rule 1: you always speak first. The kitchen never sends food nobody ordered.

Rule 2: one kitchen cooks for many tables at the same time (one server, many clients).

The waiter carries your order but never cooks: the network carries messages, it doesn't answer them.

Figure 2. The everyday picture: customer, waiter and kitchen are client, network and server.

3 Now the real thing

In the restaurant	In a network
You, the customer	the **client**: the Flipkart app on your phone
The kitchen	the **server**: a computer in a data centre that waits for requests
Your order	the **request**
The plate of food	the **response**
The waiter, carrying both ways	the **network**

The same rules hold: the client always speaks first, and one server answers many clients at once, thousands of phones in the same second.

4 Step by step: one request

1. **Ask.** You tap Search. The app writes a request: send me shoes that match *running shoes*. Its job is to say exactly what you want.
2. **Travel.** The request goes from your phone, over Wi-Fi or mobile data, through several networks, to Flipkart's server. The network carries the message; it doesn't answer it.
3. **Work.** The server finds matching shoes, with prices and photos. The thinking happens here, not on your phone.
4. **Answer.** The server sends a response: here are twenty shoes. It travels back across the networks.
5. **Show.** The app reads the response and draws it on your screen. One request, one response: a conversation of two messages.

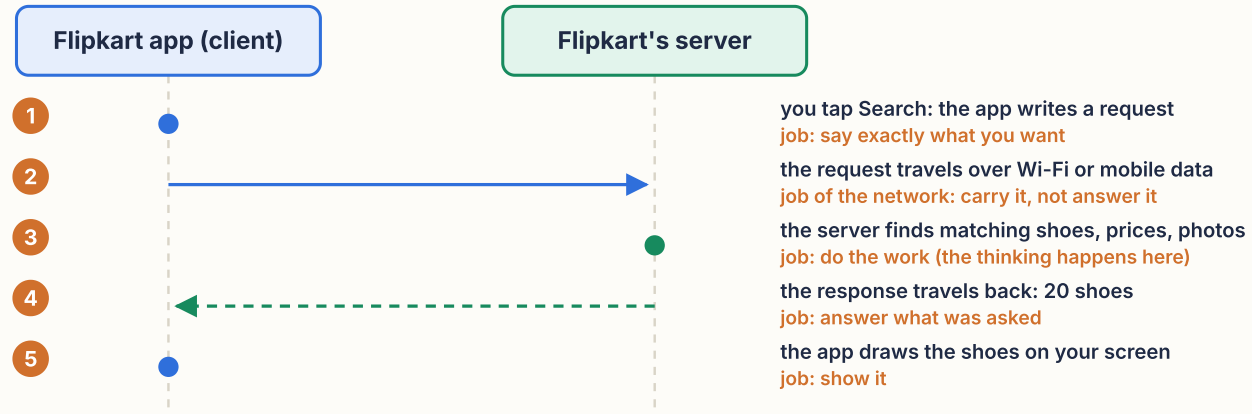


Figure 3. One request, step by step. Each step has a job: ask, carry, work, answer, show.

5 Real numbers: round trips add up

The time for a request to go and its response to come back is a **round trip**. On a mobile network, **50 ms** is common (a millisecond is a thousandth of a second). A shopping page is not one response: a **3 MB** page is made of about **40 files** (the page itself, photos, styles and code), and each file is its own request and response.

How the files are fetched	Arithmetic	Waiting on round trips
one after another	40 × 50 ms	**2,000 ms = 2 s**
six at a time	40 ÷ 6 ≈ 7 rounds; 7 × 50 ms	**350 ms**

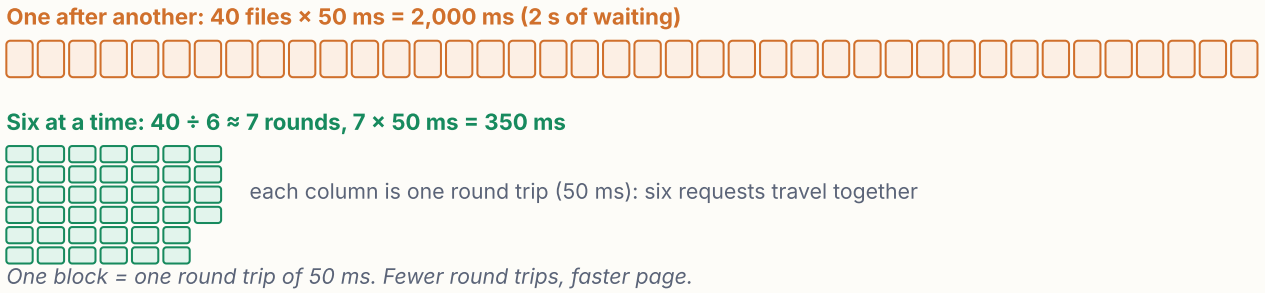


Figure 4. Real numbers: a 3 MB page of about 40 files, at 50 ms per round trip.

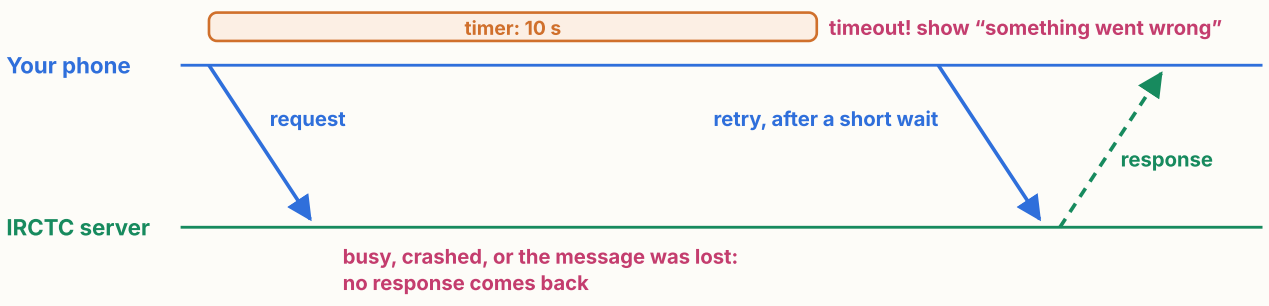
For the curious. Round trips are only half the cost; moving the bytes takes time too. 3 MB is 24 megabits (1 byte = 8 bits), so on a 20 megabit-per-second connection the data alone takes 24 ÷ 20 = 1.2 s. Browsers really do fetch several files at once (HTTP/1.1 browsers open about six connections per site; HTTP/2, lesson 23, sends many requests over one connection).

6 What goes wrong: no answer

At 10 in the morning Tatkal booking opens on IRCTC, and lakhs of phones send requests in the same minute. A busy server may answer slowly, or answer “I'm busy, try again later”. Worse, sometimes no answer comes at all: the server may have crashed, or a message was lost on the way.

The client can't wait forever. It starts a timer; if no response arrives in, say, 10 seconds, it gives up. That is a **timeout**, and the app shows “something went wrong, please try again”.

- **Timeouts**, so the app never hangs forever.
- **Retries after a short wait**, so a busy server isn't hit even harder.
- **Safe retries.** Asking twice for a train list is harmless; paying twice is not. (How systems prevent double payments comes later in the course.)
- **More servers** that share the requests, like a restaurant opening more kitchens.



A retry must be safe: asking twice for a train list is harmless, paying twice is not.

Figure 5. What goes wrong: no response comes. The client's timer runs out (a timeout), and it retries safely after a short wait.

PAUSE AND THINK

You send “hi” to a friend on WhatsApp. Who is the client? Who is the server? And is your friend's phone a server? Cover the answer and decide.

Answer. Your phone is a client: it sends a request to WhatsApp's server. The server holds the message and delivers it to your friend's phone, which is a client too. The two phones don't talk directly; they talk through the server.

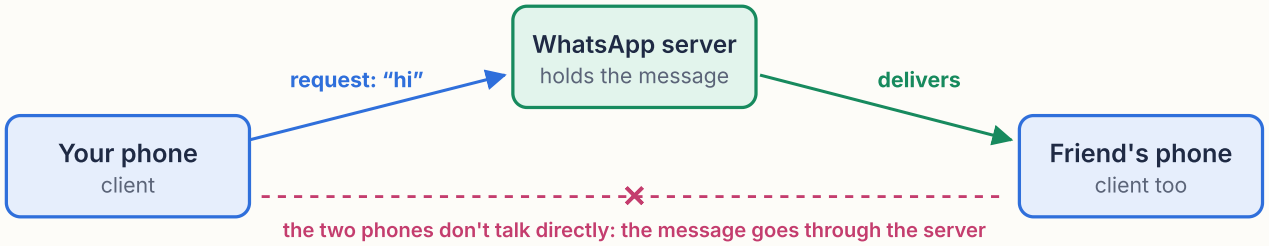


Figure 6. The pause-and-think answer: both phones are clients; the WhatsApp server sits in the middle.

TWO COMMON MISTAKES

- **“A server is a special, giant machine.”** No. A server is any computer running a program that waits for requests; even your laptop can be one. Client and server are roles: who asks, and who answers.
- **“The network understands your message.”** No. It only carries it, like a waiter who carries your order but never cooks. The meaning lives at the two ends, the client and the server.

RECAP IN THREE LINES

1. A network links devices so they can send messages.
2. A client sends a request; a server does the work and sends back a response.
3. Every round trip costs time, and when no answer comes, a timeout and a safe retry keep things moving.

7 Interview questions: what to say in 30 seconds

Q1. What is a network, and what are a client and a server? BASIC

A network is devices connected so they can exchange messages. In the client-server model the client starts the conversation with a request, and the server, a program that waits for requests, does the work and sends back a response.

Q2. Is a server a special kind of machine? BASIC

No. A server is a role: any computer running a program that waits for requests. A laptop can be a server. Client and server describe who asks and who answers, not the hardware.

Q3. A page needs 40 files and a round trip is 50 ms. How long does it wait if it fetches them one by one, or six at a time? SCENARIO

One by one: 40 × 50 ms = 2,000 ms, two seconds of waiting before moving any data. Six at a time: about 7 rounds, 7 × 50 = 350 ms. That's why designs try to cut round trips.

Q4. Tatkal booking opens and the server is overloaded. What should the client do? SCENARIO

Never wait forever: use a timeout, then retry after a short, growing wait so the busy server isn't hit even harder, and only retry requests that are safe to repeat. On the server side, add servers to share the load.

Q5. When you message a friend on WhatsApp, is your friend's phone the server? TRICKY

No. Both phones are clients. Your phone sends the message to WhatsApp's server, which holds it and delivers it to your friend's phone. The phones don't talk directly.

Q6. Why must retries be safe? TRICKY

Because a timeout doesn't tell you whether the server did the work. The request may have succeeded and only the response was lost. Asking twice for a train list is harmless; paying twice is not, so such requests need protection against duplicates.

Read more. RFC 1122 (requirements for internet hosts) and MDN's "How the web works". The full written lesson is at backendroadmap.com.

Next: SD-N-002 · Packets: why data travels in small pieces backendroadmap.com